MFD Portfolio Intelligence: Streamlit → FastAPI + React on Rooted Termux (proot)

1. Target Architecture

```
Phone (rooted, Termux)
├─ Termux native
│   ├─ tsu / su            (root helpers)
│   ├─ termux-services     (runit: keeps processes alive)
│   ├─ Termux:Boot         (autostart on reboot)
│   ├─ cloudflared         (remote access, no port-forward)
│   └─ redis-server        (optional cache)
└─ proot-distro Debian (arm64)
    ├─ FastAPI + Uvicorn   (API, port 8000)
    ├─ APScheduler worker  (separate process: mailback, NAV, BSE)
    ├─ Chromium + chromedriver (BSE Selenium)
    ├─ SQLite (WAL mode)
    └─ frontend/dist       (static React build, served by FastAPI)
```

Decisions

Item	Choice	Reason
Backend	FastAPI + SQLAlchemy 2 + Alembic	Async, auto OpenAPI docs, light on RAM
DB	SQLite (WAL)	Already used; no server to run on phone
Frontend	Vite + React + TypeScript + TanStack Query + Tailwind/shadcn	Builds to static files. No Node needed on phone. (Replaces the earlier Next.js suggestion.)
Scheduler	APScheduler in a separate process	No cron/systemd needed in proot
Cache	`cachetools` first, Redis later	Fewer moving parts
Process manager	runit via `termux-services`	Works without systemd
Remote access	Cloudflare Tunnel	Works behind mobile CGNAT
Selenium	Debian `chromium` + `chromium-driver`	Ubuntu's Chromium is a snap and fails in proot

Root advantages

- Use `chroot` instead of proot for ~2x speed (real mounts, no ptrace overhead). Keep the same Debian rootfs. Proot works as the fallback.
- `su -c` to whitelist Termux from battery optimization and Doze.
- `termux-wake-lock` keeps CPU awake.
- Bind port 80/443 directly if wanted.
- Pin the process priority: `su -c "renice -n -5 -p <pid>"`.

---

2. Five Steps

STEP 1: Environment + project skeleton + database models

Files used: `init_db.py`

Do

1. Termux:

```
pkg install proot-distro tsu termux-services termux-api cloudflared git
proot-distro install debian
proot-distro login debian
```

2. Inside Debian:

```
apt update && apt install -y python3 python3-venv python3-pip chromium chromium-driver sqlite3
python3 -m venv ~/venv && . ~/venv/bin/activate
pip install fastapi uvicorn[standard] sqlalchemy alembic pydantic-settings apscheduler python-j
```

3. Create layout:

```
mfd/
  backend/app/{main.py, api/v1/, core/, models/, schemas/, services/, workers/}
  backend/alembic/
  frontend/
```

4. `core/config.py` (env vars: `DB_PATH`, `JWT_SECRET`, `BSE_SCHEME_DIR`), `core/db.py` (engine, `PRAGMA journal_mode=WAL`, `busy_timeout=5000`).
5. Convert every `CREATE TABLE` in `init_db.py` into SQLAlchemy models in `models/`. Include `nav_schemes`, `nav_options`, `nav_history`, `admin_settings`, and the AUM recon tables. Add a new `job_runs` table: `id, task, trigger, started_at, finished_at, ok, msg`.
6. `alembic revision --autogenerate`, then `alembic stamp head` against your existing DB, so no data is lost.

Done when: `alembic upgrade head` runs clean on a copy of the production DB.

STEP 2: Extract business logic into `services/` (remove all Streamlit)

Files used: `capital_gain.py`, `trading_day_calender.py`, `nav_queries.py`, `nav_data_ingestion.py`, `nav_service.py`, `data_manager.py`, `aum_recon.py`, `cams_mailback_sync.py`, `bse_auto.py`, `email_tempate.py`, `sync_failure_log.py`

Do

File	Destination	Change
`capital_gain.py`	`services/capital_gain.py`	Copy as is (already pure).
`trading_day_calender.py`	`services/trading_calendar.py`	Copy; move holiday dict to a JSON file.
`nav_queries.py`	`services/nav_queries.py`	Replace `init_db.get_conn` with the SQLAlchemy session; rename the `no` alias (it is a SQL keyword risk).
`nav_data_ingestion.py`	`services/nav_ingest.py`	Same session swap; batch inserts with `executemany`.
`nav_service.py`	`services/nav_service.py`	Remove `@st.cache_data`, `st.session_state`.
`data_manager.py`	`services/data_manager.py`	Remove `st.*`. Parsers take a file

		object and return plain dicts.
`aum_recon.py`	`services/aum_recon.py`	Same.
`cams_mailback_sync.py`	`services/mailback.py`	Credentials from env/DB, not `st.secrets`.
`bse_auto.py`	`services/bse.py`	Keep `_do_download`, `sync_now`. Delete the unused thread helpers. Remove `webdriver_manager`; use system chromedriver only.
`email_tempate.py`	`services/email_templates.py`	Copy; fix filename typo.
`sync_failure_log.py`	`services/sync_log.py`	Copy; optionally write to a DB table too.

Cache replacement:

```
from cachetools import TTLCache
_cache = TTLCache(maxsize=512, ttl=300)
def data_version() -> int: ...   # reads admin_settings 'data_version'
```

Key every cached call with `data_version()`. The worker bumps it after each run (replaces `bump()` / `.worker_run_stamp`).

Done when: `pytest` passes for `capital_gain` (FIFO, LTCG date boundaries), `trading_calendar`, and NAV parse of a sample `NAVAll.txt`. `grep -r "import streamlit" backend/` returns nothing.

STEP 3: Build the REST API

Files used: `app.py` (as the feature list/reference), `data_manager.py`, `nav_queries.py`, `capital_gain.py`, `aum_recon.py`, `email_tempate.py`, `runtime_status.py`, `sync_failure_log.py`

Do

1. Read `app.py` page by page and list every feature. Each becomes an endpoint.
2. Routers in `api/v1/`:

Router	Endpoints
`auth`	`POST /auth/login`, `GET /auth/me`
`nav`	`GET /nav/latest`, `/nav/{isin}/history`, `/nav/previous`
`clients`	CRUD, `GET /clients/{id}/portfolio`, `/transactions`
`capital_gains`	`GET /clients/{id}/capital-gains`, `POST /capital-gains/hypothetical`
`aum`	`GET /aum`, `/aum/reconciliation`
`uploads`	`POST /uploads/cams`, `/uploads/kfin`, `/uploads/bse` (save file, enqueue job, return `job_id`)
`reports`	`POST /reports/valuation`, `/reports/capital-gain` (PDF), `POST /reports/{id}/email`
`admin`	`GET /admin/sync-status`, `/admin/failures`, `/admin/settings`, `POST /admin/jobs/{name}/run`, `GET /admin/data-version`

3. Pydantic response models for every endpoint. Numbers as numbers; no formatted strings.
4. `runtime_status.py` is replaced by the `job_runs` table. `sync_failure_log.py` serves `/admin/failures`.
5. Auth: JWT, bcrypt password, one admin user seeded by CLI.
6. Mount static frontend last: `app.mount("/", StaticFiles(directory="frontend/dist", html=True))`.

Done when: `uvicorn app.main:app --host 0.0.0.0 --port 8000` runs and `/docs` shows all endpoints. Compare 5 portfolio outputs against Streamlit values.

STEP 4: Worker + scheduler (replaces cron, watchdog, launcher)

Files used: `background_worker.py`, `worker_launcher.py`, `worker_watchdog.py`, `bse_auto.py`, `runtime_status.py`, `sync_failure_log.py`

Do

1. `workers/main.py` (own process) with APScheduler:

   ```
   sched.add_job(run_all, "cron", hour="11,15", id="main", max_instances=1, coalesce=True, misfire
   ```

   - `max_instances=1` replaces the `fcntl` lock.
   - `coalesce` + `misfire_grace_time` replace `worker_watchdog.py` (missed runs catch up automatically on restart).
2. Move `task_mailback`, `task_nav_live`, `task_nav_previous`, `task_bse` from `background_worker.py` into `workers/tasks.py`. Keep the `admin_settings` enable flags. Note: `task_bse` is defined but **not** in the `run_all` task list; add it if BSE should run on schedule.
3. Each task writes a `job_runs` row (replaces `runtime_status.py` and the stamp file). After `run_all`, bump `data_version`.
4. "Run now" button: `POST /admin/jobs/{name}/run` inserts a `pending` row in `job_runs`; the worker polls it every 10s. This replaces `worker_launcher.py` (no subprocess spawn from the API).
5. Keep the Streamlit shim only if some module still imports `st`. After Step 2 it should be deleted.
6. Keep `TZ=Asia/Kolkata` in the service env.

Done when: killing the worker and restarting it after a 11:00 slot triggers a catch-up run; the manual endpoint runs a task and shows it in `job_runs`.

STEP 5: Frontend + hosting on the phone

Files used: `app.py` (UI layout reference), `theme_patch.py` (colors), `email_tempate.py` (preview), plus the OpenAPI schema from Step 3

Frontend (build on a laptop, not the phone)

```
npm create vite@latest frontend -- --template react-ts
npm i @tanstack/react-query react-router-dom recharts axios
npx shadcn@latest init
```

- Generate typed client: `npx openapi-typescript http://phone:8000/openapi.json -o src/api/schema.ts`
- Theme: move the `THEME_CSS` palettes from `theme_patch.py` into Tailwind CSS variables (`--bg #0e1117`, `--card #161b22`, `--accent #58a6ff` for dark; `#fff / #f6f8fa / #2563eb` for light). Toggle with a

dark class; delete THEME_WATCHER_JS .

- .metric-card and .aum-card* become <KpiCard variant="primary|success|info|warning">.
- Pages: Login, Dashboard (AUM + KPIs), Clients, Portfolio, Capital Gains (with hypothetical redemption form), Reports, Uploads, Admin (sync status, failures, settings, run-now).
- TanStack Query: staleTime NAV 5 min; poll /admin/data-version every 60s and invalidate all queries on change.
- npm run build , then copy dist/ to mfd/frontend/dist on the phone (scp or termux-setup-storage).

Hosting (Termux)

1. Run API and worker as runit services. In Termux:

   ```
   mkdir -p $PREFIX/var/service/mfd-api $PREFIX/var/service/mfd-worker
   ```

   mfd-api/run :

   ```
   #!/data/data/com.termux/files/usr/bin/sh
   exec proot-distro login debian --shared-tmp -- bash -lc 'cd ~/mfd/backend && . ~/venv/bin/activ
   ```

   mfd-worker/run : same, with python -m app.workers.main . chmod +x both, then sv-enable mfd-api mfd-worker .

2. Autostart on reboot: install the Termux:Boot app, then ~/.termux/boot/start.sh :

   ```
   #!/data/data/com.termux/files/usr/bin/sh
   termux-wake-lock
   sv-enable mfd-api mfd-worker
   ```

3. Root tweaks (run once as root):

   ```
   su -c "dumpsys deviceidle whitelist +com.termux"
   su -c "settings put global app_standby_enabled 0"
   ```

   Optional speedup: switch the proot-distro login line to a chroot of the same rootfs via su -c .

4. Remote access: cloudflared tunnel login , create a tunnel pointing to http://localhost:8000 , run it as a third runit service. Put Cloudflare Access (email OTP) in front on top of JWT.
5. Backups: nightly sqlite3 mfd.db ".backup /sdcard/backup/mfd_$(date +%F).db" via APScheduler or crond . Sync the folder off-device.

Done when: after a phone reboot, the dashboard loads from your public tunnel URL with no manual action, and the 15:00 worker run appears in job_runs .

---

3. Phone-Specific Gotchas

- **RAM:** run --workers 1 . Chromium is the heaviest piece; it runs only during the BSE task. Keep --headless=new --no-sandbox --disable-dev-shm-usage (already in bse_auto.py).
- **SQLite:** WAL mode plus one writer. Keep the worker's long ingests in batched transactions to avoid blocking API writes.
- **Storage:** keep the DB and Reports/ on internal storage (ext4), not /sdcard (FUSE, slow, no file locking).
- **Proot:** fcntl works, but avoid heavy native builds. Prefer wheels (pandas , bcrypt have arm64 wheels).
- **Security:** bind the API to 127.0.0.1 ; only the tunnel exposes it. Store Gmail credentials in env vars or an encrypted table, never in the repo.
- **Thermals:** a phone running 24/7 on charge degrades the battery. Use a charge limiter (root: su -c "echo 80 > /sys/class/power_supply/battery/charge_control_limit" , path varies by device) or remove the

battery.

4. File Disposition Summary

File	Fate
`init_db.py`	→ models + Alembic (Step 1)
`capital_gain.py`, `trading_day_calender.py`, `email_tempate.py`	Copy to services (Step 2)
`nav_queries.py`, `nav_data_ingestion.py`, `nav_service.py`, `data_manager.py`, `aum_recon.py`, `cams_mailback_sync.py`, `bse_auto.py`, `sync_failure_log.py`	Refactor to services (Step 2)
`app.py`	Feature reference → API + React pages (Steps 3, 5)
`runtime_status.py`	Replaced by `job_runs` table (Steps 3, 4)
`background_worker.py`	Tasks → `workers/tasks.py` (Step 4)
`worker_launcher.py`, `worker_watchdog.py`	Deleted; replaced by APScheduler + job queue (Step 4)
`theme_patch.py`	Colors → Tailwind tokens, then deleted (Step 5)